

به نام خدا



پروژه درس جبر خطی - ترم ۴۰۴۲

Indexing Semantic Latent

استاد : دکتر نیکان

اعضای گروه: علی کارخانه - مصطفی شیبانی - حسین نادری - مرتضی رحیمی

سوالات عملی:

سوال ۷ – بارگذاری و پیش‌پردازش متن (حذف علائم نگارشی و کوچک‌سازی حروف)
در پروژه‌های بازیابی اطلاعات و نمایه سازی پنهان متن (LSI)، پیش‌پردازش متن یکی از حیاتی‌ترین مراحل است. هدف از این کار، حذف اطلاعات بیهوده (Noise) و یکدست‌سازی کلمات است تا محاسبات برداری دقیق‌تر انجام شوند. کد زیر فرآیند تمیزکاری متون را انجام می‌دهد:

```
1 # =====
2 # Q7 - Load + preprocess (lowercase, remove punctuation)
3 # =====
4 def preprocess(text):
5     text = text.lower()
6     text = re.sub(r'[^a-z\s]', ' ', text)
7     text = re.sub(r'\s+', ' ', text).strip()
8     return text
9
10 def q7_load_and_preprocess():
11     df = pd.read_csv(DATA_PATH)
12     print("Q7 | Dataset shape:", df.shape)
13     df['clean_text'] = df['Text'].apply(preprocess)
14     return df
```

توضیح عملکرد متدها و توابع:

۱. تابع `preprocess(text)`: این تابع یک متن خام را به عنوان ورودی دریافت کرده و مراحل زیر را روی آن اعمال می‌کند تا متنی یکدست ایجاد شود:

- **تبدیل به حروف کوچک (`text.lower()`)**: تمام حروف بزرگ انگلیسی به حروف کوچک تبدیل می‌شوند. این کار باعث می‌شود کلماتی مانند "Data" و "data" یکسان دیده شوند و ابعاد ماتریس نهایی بیهوده بزرگ نشود.
- **حذف کاراکترهای غیرالفبایی**: با استفاده از عبارات منظم (Expressions Regular)، هر کاراکتری که جزء حروف الفبای انگلیسی (a-z) یا فاصله (\s) نباشد، با یک فاصله معمولی جایگزین می‌شود. این کار علائم نگارشی نظیر ویرگول، نقطه، علامت سوال و همچنین اعداد را کاملاً حذف می‌کند.
- **حذف فاصله‌های اضافه**: حذف علائم نگارشی در مرحله قبل ممکن است چندین فاصله متوالی ایجاد کرده باشد. الگوی `\s+` تمام فاصله‌های متوالی را به یک تک‌فاصله تبدیل می‌کند و متد `strip()` نیز فاصله‌های خالی ابتدا و انتهای متن را پاک می‌کند.

۲. تابع `q7_load_and_preprocess()`: این تابع وظیفه مدیریت جریان داده را بر عهده دارد:

- ابتدا مجموعه داده را از مسیر `DATA_PATH` به صورت یک دیتافریم پانداس بارگذاری می‌کند.
- با چاپ `df.shape`، ابعاد داده‌ها (تعداد کل اسناد و ویژگی‌ها) را جهت بررسی صحت بارگذاری نمایش می‌دهد.
- در نهایت با استفاده از متد `apply()`، تابع تمیزکاری متنی را روی تک‌تک سطرهای ستون `'Text'` اعمال کرده و نتیجه را در ستون جدیدی به نام `'clean_text'` ذخیره می‌کند.

۳. **خروجی اجرای تابع و تحلیل ساختار داده‌ها**: پس از فراخوانی تابع بارگذاری و پیش‌پردازش متنی، پیش‌نمایشی از دیتافریم خروجی به همراه ابعاد آن تولید می‌شود:

- **ابعاد مجموعه داده** (`Dataset shape: (2225, 2)`): خروجی نشان می‌دهد که دیتاست اولیه دارای ۲۲۲۵ سطر (سند متنی) و ۲ ستون اصلی است. پس از اجرای تابع و اضافه شدن ستون تمیزشده، تعداد ستون‌ها به ۳ افزایش می‌یابد (`2225 rows × 3 columns`).

• بررسی ساختار ستون‌ها:

۱. **ستون Text**: شامل متن خام اسناد است که در آن علائم نگارشی، کاراکترهای خط جدید (`\n`) و حروف بزرگ انگلیسی به همان شکل اولیه دیده می‌شوند.
۲. **ستون Label**: شامل برچسب‌های عددی (مثلاً دسته ۰ برای سیاست و دسته ۴ برای تجارت) است که دسته‌بندی موضوعی اسناد را مشخص می‌کند.

Q7 | Dataset shape: (2225, 2)

	Text	Label	clean_text
0	Budget to set scene for election\n\n Gordon B...	0	budget to set scene for election gordon brown ...
1	Army chiefs in regiments decision\n\n Militar...	0	army chiefs in regiments decision military chi...
2	Howard denies split over ID cards\n\n Michael...	0	howard denies split over id cards michael howa...
3	Observers to monitor UK election\n\n Minister...	0	observers to monitor uk election ministers wil...
4	Kilroy names election seat target\n\n Ex-chat...	0	kilroy names election seat target ex chat show...
...	...	...	...
2220	India opens skies to competition\n\n India wi...	4	india opens skies to competition india will al...
2221	Yukos bankruptcy 'not US matter'\n\n Russian ...	4	yukos bankruptcy not us matter russian authori...
2222	Survey confirms property slowdown\n\n Governm...	4	survey confirms property slowdown government f...
2223	High fuel prices hit BA's profits\n\n British...	4	high fuel prices hit ba s profits british airw...
2224	US trade gap hits record in 2004\n\n The gap ...	4	us trade gap hits record in the gap between us...

2225 rows x 3 columns

شکل ۱: خروجی نهایی دیتافریم پس از انجام عملیات پیش‌پردازش متنی

۳. **ستون جدید clean_text:** خروجی نهایی حاصل از اعمال تابع preprocess است. همان‌طور که در تصویر مشخص است، تمامی حروف بزرگ به حروف کوچک تبدیل شده‌اند، کاراکترهای کنترلی مانند \n و علائم نگارشی کاملاً حذف شده و کلمات صرفاً با یک تک‌فاصله از یکدیگر جدا شده‌اند تا برای پردازش‌های بعدی آماده باشند.

سوال ۸ – استخراج ۳۰ کلمه پرتکرار و رسم نمودار میله‌ای
در این بخش، هدف شناسایی و استخراج ۳۰ کلمه پرتکرار موجود در کل اسناد با امکان فعال یا غیرفعال‌سازی فیلتر کلمات توقف (Stopwords) است. کد مربوط به این بخش به صورت زیر است:

```

1 # =====
2 # Q8 - Top 30 most frequent words + bar chart
3 # =====
4 def q8_top30_words(df, fname, stopwords=False):
5     if stopwords:
6         stopwords = ENGLISH_STOP_WORDS
7     else:
8         stopwords = {}
9
10    stop_words_set = set(stopwords)
11
12    all_words = []
13    for text in df['clean_text']:
14        words = [w for w in text.split() if w not in stop_words_set and len(w) > 1]
15        all_words.extend(words)
16
17    counter = Counter(all_words)
18    top30 = counter.most_common(30)
19    print("\nQ8 | Top 30 frequent words:", top30)
20
21    words, counts = zip(*top30)
22    fig, ax = plt.subplots(figsize=(14, 6))
23    ax.bar(words, counts, color='steelblue')
24    plt.xticks(rotation=60, ha='right')
25    ax.set_title('Top 30 Most Frequent Words (all documents)')
26    ax.set_ylabel('Frequency')
27    plt.tight_layout()
28    % plt.savefig(f'media/{fname}.png', dpi=130)
29    plt.close(fig)
30
31    return top30

```

توضیح عملکرد متدها و توابع:

۱. **منطق شرطی کلمات توقف (stopwords و stop_words_set):** تابع در این نسخه منعطفتر طراحی شده است و پارامتر stopwords را به عنوان ورودی می‌پذیرد:

- اگر مقدار stopwords برابر با True پاس داده شود، کلمات توقف استاندارد انگلیسی (ENGLISH_STOP_WORDS) به عنوان فیلتر فعال می‌شوند.
- در غیر این صورت (حالت False)، یک دیکشنری خالی {} تخصیص داده می‌شود که به معنی عدم فیلتر کردن کلمات توقف است.
- کلمات توقف در نهایت به یک ساختار مجموعه (set) تبدیل می‌شوند؛ زیرا جستجو در ساختار مجموعه در پایتون از مرتبه زمانی $O(1)$ بوده و سرعت پردازش را بسیار افزایش می‌دهد.

۲. جمع‌آوری و فیلتر کلمات:

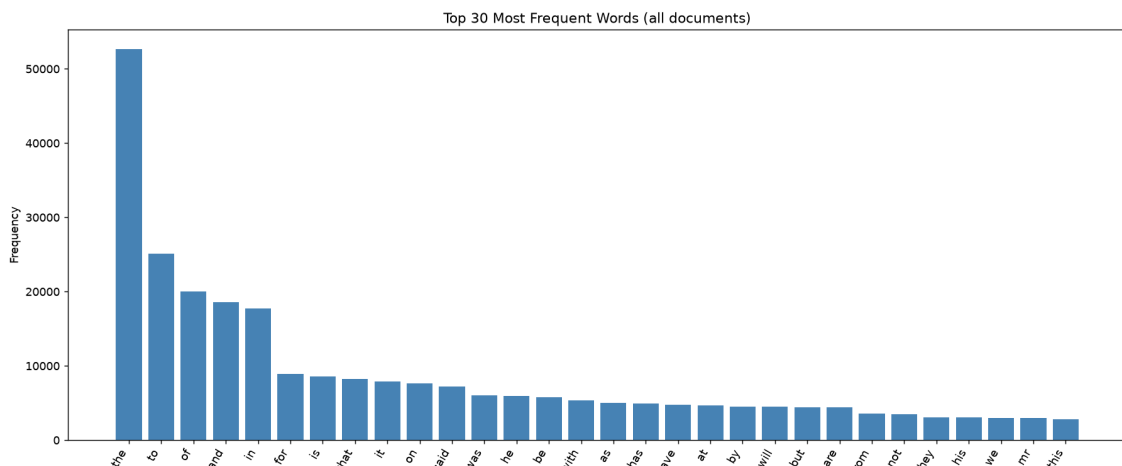
- با پیمایش ستون متون تمیزشده (clean_text)، هر متن با متد split() به کلمات سازنده‌اش شکسته می‌شود.
- کلماتی که در مجموعه‌ی کلمات توقف تعریف‌شده وجود نداشته باشند (w not in stop_words_set) و طول آن‌ها بزرگتر از یک حرف باشد ($\text{len}(w) > 1$)، فیلتر شده و به لیست نهایی اضافه می‌گردند.

۳. **شمارش و استخراج ۳۰ کلمه برتر:** کلاس Counter فرکانس تکرار کلمات را محاسبه کرده و با متد most_common(30)، ۳۰ کلمه با بیشترین تکرار را به صورت لیستی از توپل‌های (کلمه، فرکانس) باز می‌گرداند.

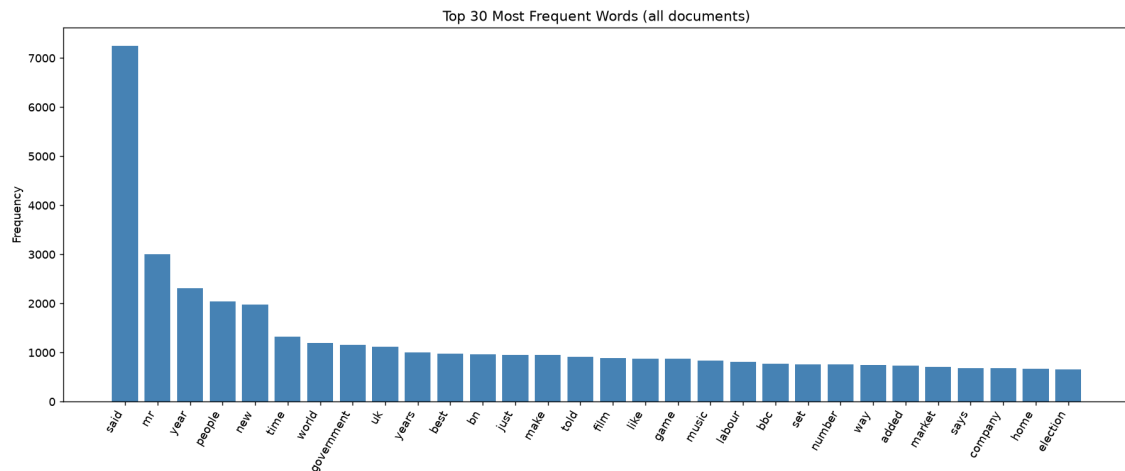
۴. رسم و مدیریت نمودار میله‌ای:

- با دستور zip(*top30) داده‌ها باز شده و به دو بخش کلمات و فرکانس‌ها تقسیم می‌شوند تا در نمودار میله‌ای به کار روند.
- نمودار میله‌ای با رنگ آبی فولادی (steelblue) رسم شده و برجسب‌های محور افقی جهت جلوگیری از هم‌پوشانی ۶۰ درجه چرخانده می‌شوند.
- نام فایل ذخیره‌سازی نمودار به صورت پویا با استفاده از متغیر ورودی fname تعیین شده است (که در کدهای فعلی جهت صرفه‌جویی در پردازش کامنت شده است) و در نهایت با plt.close() پنجره ترسیم بسته می‌شود تا حافظه آزاد گردد.

۵. **بررسی تجربی و مقایسه نمودارها (نقش حیاتی کلمات توقف):** برای درک عمیق‌تر تأثیر پیش‌پردازش، تابع فوق در دو حالت اجرا شده و نتایج آن در قالب دو نمودار زیر ذخیره و تحلیل شده است:



شکل ۲: ۳۰ کلمه پرتکرار بدون حذف کلمات توقف (غلبه کامل حروف ربط و اضافه)



شکل ۳: ۳۰ کلمه پرتکرار پس از حذف کلمات توقف (نماین شدن کلمات کلیدی موضوعی)

• **تحلیل حالت اول (بدون حذف کلمات توقف – شکل ۲):** همان‌طور که در نمودار اول مشاهده می‌شود، کلماتی مانند “the” (با بیش از ۵۰,۰۰۰ تکرار)، “to”، “of” و “and” کاملاً بر داده‌ها غلبه کرده‌اند. این کلمات صرفاً نقش ساختاری در زبان انگلیسی دارند و کوچک‌ترین اطلاعاتی درباره موضوع اصلی سند (مانند ورزشی، سیاسی یا علمی بودن آن) به ما نمی‌دهند.

• **تحلیل حالت دوم (با حذف کلمات توقف – شکل ۳):** در نمودار دوم، با فعال‌سازی پارامتر `stopwords=True` و فیلتر کردن این کلمات بیهوده، سقف فرکانس کلمات به زیر ۸,۰۰۰ تکرار کاهش یافته است. با حذف نویزهای زبانی، کلمات معنادار و موضوعی‌تری مانند “government” (دولت)، “film” (فیلم)، “game” (بازی)، “music” (موسیقی) و “election” (انتخابات) ظاهر شده‌اند که به ما در دسته‌بندی و تحلیل محتوایی اسناد کمک شایانی می‌کنند.

سوال ۹ – رسم ابر کلمات اختصاصی براساس وزن‌دهی TF-IDF

در این بخش، هدف نمایش بصری کلمات کلیدی اسناد با استفاده از یک ابر کلمات (Word Cloud) اختصاصی است. از آنجایی که پکیج پیش‌فرض `wordcloud` در حالت آفلاین در دسترس نبوده، پیاده‌سازی به صورت دستی و با استفاده از توانمندی‌های ترسیم کتابخانه `matplotlib` انجام گرفته است. معیار انتخاب کلمات، وزن مجموع فرکانس سند معکوس (TF-IDF) آن‌هاست. کد بهینه‌شده به صورت زیر است:

```
1 # =====
2 # Q9 - Word cloud (top 20 words by TF-IDF), custom-rendered
3 # (wordcloud package unavailable offline -> manual matplotlib rendering)
4 # =====
5 def q9_wordcloud(df, fname):
6     extra_stop = ['said', 'mr', 'told', 'year', 'years', 'new', 'time', 'best', 'world']
7     stop_words = list(ENGLISH_STOP_WORDS.union(extra_stop))
8
9     vectorizer = TfidfVectorizer(stop_words=stop_words, max_df=0.9, min_df=2)
10    tfidf = vectorizer.fit_transform(df['clean_text'])
11    scores = np.asarray(tfidf.sum(axis=0)).ravel()
12    vocab = vectorizer.get_feature_names_out()
13
14    top_idx = np.argsort(scores)[::-1][:20]
15    top_words = [(vocab[i], scores[i]) for i in top_idx]
16
17    random.seed(7)
18    fig, ax = plt.subplots(figsize=(12, 8))
19    ax.axis('off')
20
21    max_score = max(s for _, s in top_words)
22    min_score = min(s for _, s in top_words)
23    placed = []
24
25    def overlaps(x, y, w, h, placed):
```

```

26         for (px, py, pw, ph) in placed:
27             if abs(x - px) < (w + pw) / 2 * 0.75 and abs(y - py) < (h + ph) / 2 *
0.75:
28                 return True
29         return False
30
31     for word, score in top_words:
32         size = 16 + 55 * (score - min_score) / (max_score - min_score)
33
34         for _ in range(300):
35             x = random.uniform(0.1, 0.9)
36             y = random.uniform(0.1, 0.9)
37             w = len(word) * size * 0.011
38             h = size * 0.02
39             if not overlaps(x, y, w, h, placed):
40                 placed.append((x, y, w, h))
41                 break
42
43         color = plt.cm.tab20(random.random())
44         ax.text(x, y, word, fontsize=size, ha='center', va='center', color=color,
45               fontweight='bold', fontfamily='sans-serif', transform=ax.transAxes)
46
47     ax.set_title('Top 20 Words Wordcloud', fontsize=16)
48     plt.tight_layout()
49     % plt.savefig(f'media/{fname}.png', dpi=130)
50     plt.close(fig)
51
52     return top_words

```

توضیح عملکرد متدها و تغییرات جدید:

۱. **توسعه‌ی کلمات توقف (stop_words):** مجموعه‌ای از کلمات عمومی انگلیسی (ENGLISH_STOP_WORDS) با کلمات پر تکرار کم‌اهمیت دیگر که در متغیر extra_stop تعریف شده‌اند، ادغام (union) می‌شوند. این کار باعث تفکیک کلمات موضوعی باارزش از کلمات تکراری زبان می‌شود.

۲. **بردارسازی با معیار TF-IDF (TfidfVectorizer):** یک مدل بردار ساز با حذف کلمات توقف تعریف می‌شود. پارامتر max_df=0.9 کلماتی را که در بیش از ۹۰ درصد اسناد تکرار شده‌اند حذف می‌کند و min_df=2 کلماتی را که کمتر از ۲ بار در کل پیکره متنی تکرار شده‌اند نادیده می‌گیرد. در نهایت، با متد fit_transform() ماتریس ویژگی‌های اسناد محاسبه می‌شود.

۳. **استخراج کلمات کلیدی برتر براساس مجموع امتیازها:** با محاسبه‌ی مجموع امتیازات ماتریس در طول ستون‌ها (tfidf.sum(axis=0)) و مرتب‌سازی نزولی اندیس‌ها با استفاده از np.argsort، ۲۰ کلمه‌ی برتر که بیشترین وزن مجموع را در کل پیکره متنی دارند استخراج می‌شوند.

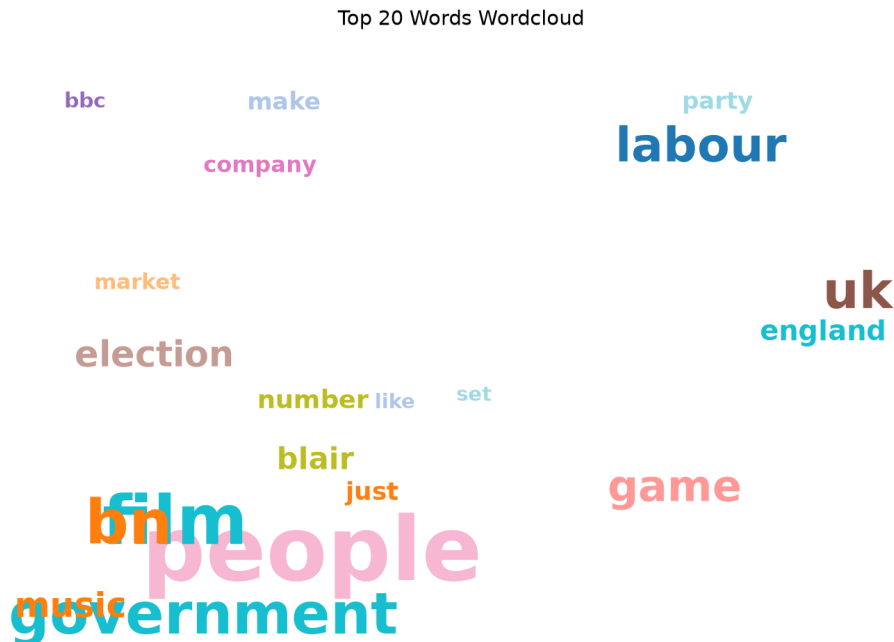
۴. **الگوریتم چیدمان غیرهم‌پوشان (overlaps):** برای جلوگیری از هم‌پوشانی و روی هم افتادن کلمات در تصویر خروجی، تابع کمکی overlaps تعریف شده است. این تابع با تخمین زدن کادر پیرامونی هر کلمه بر اساس طول کلمه و ابعاد فونت، فاصله‌ی موقعیت کلمه‌ی جدید با کلماتی که پیش‌تر جایگذاری شده‌اند را بررسی کرده و در صورت تداخل، موقعیت تصادفی جدیدی را امتحان می‌کند.

۵. شبیه‌سازی ابر کلمات و بهبودهای بصری جدید:

- **مقیاس‌دهی پویا به فونت کلمات:** اندازه‌ی قلم (size) هر کلمه متناسب با فرمول نگاشت خطی بین بازه‌ی ۱۶ الی ۷۱ پیکسل بر اساس امتیاز TF-IDF آن تعیین می‌شود تا کلمات با ارزش‌تر، بزرگ‌تر رندر شوند.
- **رنگ‌آمیزی با پالت متمایز tab20:** رنگ هر کلمه به طور تصادفی از پالت رنگی استاندارد و متمایز tab20 تخصیص می‌یابد تا کنتراست و خوانایی کلمات افزایش یابد.
- **بهبود فونت و چیدمان:** فونت نوشته‌ها به حالت بدون سریف (sans-serif) و ضخیم (bold) تغییر کرده است.

• ذخیره‌سازی پویا: تصویر نهایی با قالب پویای نام فایل خروجی (fname) در مسیر media/ آماده‌ی ذخیره‌سازی است.

۶. تحلیل بصری و بررسی خروجی ابر کلمات جدید: تصویر زیر خروجی نهایی حاصل از اجرای تابع بر روی ۲۰ کلمه‌ی برتر با معیار وزن‌دهی TF-IDF را با پالت رنگی متمایز نشان می‌دهد:



شکل ۴: ابر کلمات اختصاصی رسم‌شده برای ۲۰ کلمه برتر بر اساس امتیاز مجموع TF-IDF

• ابعاد بصری و وزن‌دهی معنایی کلمات: همان‌طور که در شکل ۴ مشخص است، کلماتی نظیر “people” (به رنگ صورتی)، “government” (به رنگ فیروزه‌ای)، “labour” (به رنگ آبی تیره) و “film” با فونت بسیار بزرگ‌تری رندر شده‌اند. این اندازه بزرگ نشان‌دهنده‌ی امتیاز بالا و مجموع وزن TF-IDF بالای آن‌ها در اسناد است. یعنی این واژه‌ها نه تنها تکرار قابل‌قبولی دارند، بلکه به شدت متمایزکننده‌ی موضوعات اصلی (مانند سیاست، سینما و اقتصاد) هستند.

• تمایز موضوعی آشکار: برخلاف نمودارهای فرکانس ساده، در این تصویر کلماتی مانند “election” (انتخابات)، “blair” (بلر - نخست‌وزیر سابق بریتانیا)، “game” (بازی)، “market” (بازار)، “music” (موسیقی) و “company” (شرکت) به وضوح دیده می‌شوند. این توزیع کلمات نشان می‌دهد که پیکره متنی ما به طور مشخص شامل موضوعاتی پیرامون سیاست بریتانیا، اخبار بازار و تجارت، دنیای سرگرمی (موسیقی/بازی/فیلم) و تکنولوژی است.

• تأیید کارکرد فیلترهای بالاگذر و پایین‌گذر: موفقیت الگوریتم در این است که کلمات بسیار عمومی مانند “year”، “new” یا کلمات توقف انگلیسی استاندارد که پیش‌تر در واژه‌نامه‌ی stop_words قرار گرفتند، به کلی فیلتر شده‌اند و در ابر کلمات حضور ندارند. این فرآیند بستری عالی و با کیفیت را برای مرحله‌ی بعدی یعنی ساخت ماتریس برداری اسناد ایجاد می‌کند.

سوال ۱۰ – ساخت ماتریس کیسه کلمات (BoW) با واژگان سفارشی و تقسیم‌بندی داده‌ها
در این مرحله، هدف ساخت ماتریس کیسه کلمات (Bag-of-Words) با استفاده از یک واژگان مشخص واژه‌نامه است که از فایل خارجی words.csv بارگذاری می‌شود. در نهایت داده‌ها به دو بخش مجموعه‌ی کاری (آموزش) و مجموعه‌ی آزمون نگهداشت تقسیم می‌شوند. کد مربوطه به صورت زیر است:

```
1 # =====  
2 # Q10 - Bag-of-Words matrix using words.csv vocabulary  
3 # =====  
4 def q10_bow_matrix(df):  
5     words_df = pd.read_csv('words.csv', header=None)  
6     vocabulary_list = words_df[0].astype(str).str.lower().tolist()
```

```

7 vectorizer = CountVectorizer(vocabulary=vocabulary_list)
8 bow = vectorizer.fit_transform(df['clean_text'])
9 bow_matrix = bow.toarray()
10
11
12 print(f"\nQ10 | BoW matrix shape: {bow_matrix.shape} (docs x |vocab|={len(
13 vocabulary_list)})")
14
15 return bow_matrix
16
17 def split_2000_225(df, bow_matrix):
18     """Split literally: first 2000 rows = working set, last 225 = held-out test."""
19     df_train = df.iloc[:2000].reset_index(drop=True)
20     df_test = df.iloc[2000:].reset_index(drop=True)
21     bow_train = bow_matrix[:2000]
22     bow_test = bow_matrix[2000:]
23     print("Split | train:", df_train.shape, bow_train.shape,
24           "| test:", df_test.shape, bow_test.shape)
25     print("Split | NOTE: dataset is label-sorted, so the literal last-225 test rows
26           "
27           "are ALL label 4 (Business). See shuffled version in Q20 for a fair "
28           "per-category evaluation.")
29     return df_train, df_test, bow_train, bow_test

```

توضیح عملکرد متدها و تغییرات جدید:

۱. بارگذاری و آماده‌سازی پویا واژگان (Vocabulary):

- `pd.read_csv= ('words.csv', header one)`: واژه‌های مرجع مستقیماً از فایل `words.csv` بارگذاری می‌شوند. از آنجا که این فایل فاقد ردیف هدر (عنوان ستون) است، پارامتر `header=None` تنظیم شده است.
- یکدست‌سازی واژگان (`vocabulary_list`): با دستور `astype(str).str.lower().tolist()` تمامی کلمات موجود در ستون اول فایل به نوع رشته تبدیل شده، حروف آن‌ها کوچک می‌شود و در نهایت به یک لیست معمولی پایتون تبدیل می‌گردند. این کار باعث تطابق کامل و بدون خطای واژگان با متون پیش‌پردازش شده می‌شود.

۲. ساخت ماتریس کیسه کلمات (CountVectorizer):

- محدودسازی ستون‌ها به واژگان سفارشی: با پاس دادن پارامتر `vocabulary` (برابر با `vocabulary_list`) به کلاس `CountVectorizer`، مدل را موظف می‌کنیم که ماتریس را صرفاً بر اساس کلمات این لیست بسازد. هر کلمه‌ای در اسناد که در این لیست نباشد، نادیده گرفته خواهد شد.
- تبدیل به آرایه متراکم (`toarray()`): خروجی متد `fit_transform` یک ماتریس خلوت است که برای انجام راحت‌تر محاسبات جبر خطی بعدی (مانند تجزیه مقادیر منفرد) با متد `toarray()` به یک آرایه دو بعدی استاندارد و متراکم NumPy تبدیل می‌شود. ابعاد این ماتریس برابر با $2225 \times M$ خواهد بود که در آن M تعداد کلمات واژه‌نامه است.

۳. تقسیم‌بندی داده‌ها به مجموعه‌های آموزش و آزمون: تابع `split_2000_225` داده‌های ورودی را به دو بخش افقی تقسیم می‌کند:

- مجموعه آموزش (`train`): شامل ۲۰۰۰ سطر اول اسناد و بردارهای کیسه کلمات متناظر با آن‌هاست.
- مجموعه آزمون (`test`): شامل ۲۲۵ سطر پایانی به عنوان داده‌های آزمون مجزا و دست‌نخورده است.
- نکته مهم توزیع دسته‌ها: به دلیل مرتب بودن اولیه مجموعه داده بر اساس برچسب‌ها، ۲۲۵ سطر پایانی همگی دارای برچسب شماره ۴ (تجارت) هستند. این تقسیم‌بندی صرفاً یک پیاده‌سازی گام‌به‌گام اولیه است و جابجایی تصادفی داده‌ها (`Shuffle`) در بخش‌های بعدی برای ارزیابی نهایی به کار گرفته خواهد شد.


```
Q10 | BoW matrix shape: (2225, 53) (docs x |vocab|=53)
```

```
array([[0, 0, 1, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0],
       ...,
       [0, 0, 0, ..., 0, 0, 0],
       [0, 0, 1, ..., 0, 0, 0],
       [0, 0, 0, ..., 0, 0, 0]], shape=(2225, 53))
```

شکل ۵: نمای کلی از ماتریس فرکانسی کیسه کلمات (BoW) و ابعاد آن در خروجی مفسر

۴. خروجی فرآیند بردارسازی و تحلیل ساختار ماتریس BoW: پس از اجرای تابع، آرایه خروجی و ابعاد به دست آمده به شکل زیر نمایش داده می‌شود:

- **بررسی ابعاد ماتریس** ((shape=(2225, 53)): خروجی نشان می‌دهد که ماتریس حاصل دارای ۲۲۲۵ سطر (تعداد کل اسناد) و ۵۳ ستون است. این ابعاد دقیقاً تایید می‌کند که تعداد واژه‌های استخراج‌شده از فایل words.csv برابر با ۵۳ کلمه کلیدی متمایز بوده است. بنابراین هر سند متنی به یک بردار ویژگی ۵۳ بُعدی تصویر می‌شود.
- **تحلیل ساختار عددی ماتریس خلوت (Sparse Matrix)**: همان‌طور که در آرایه‌ی خروجی شکل ۵ مشهود است، بخش زیادی از درایه‌های ماتریس مقدار صفر (0) دارند. این ویژگی به دلیل ماهیت اسناد متنی است؛ چرا که در هر سند متنی کوتاه، تنها تعداد انگشت‌شماری از واژگان ۵۳ گانه استفاده شده‌اند. حضور مقادیری مانند 1 در درایه‌ها نمایانگر دفعات تکرار آن واژه‌ی خاص از واژه‌نامه در سند متناظر است. این ساختار صفر و یک فرکانسی، پایه‌ی ریاضی مراحل بعدی تحلیل‌های جبر خطی پروژه را شکل می‌دهد.

بخش افراز مجموعه داده (Partitioning Dataset) – تقسیم ساده در مقابل تقسیم تصادفی

در این بخش، هدف تقسیم مجموعه داده به دو بخش آموزش (Train) با ابعاد ۲۰۰۰ نمونه و آزمون (Test) با ابعاد ۲۲۵ نمونه است. برای این کار دو تابع با رویکردهای متفاوت پیاده‌سازی شده است که کد آن‌ها به صورت زیر است:

```
1 # =====
2 # Dataset Partitioning (Simple vs Shuffled Split)
3 # =====
4 def split_2000_225(df, bow_matrix):
5     """Split literally: first 2000 rows = working set, last 225 = held-out test."""
6     df_train = df.iloc[:2000].reset_index(drop=True)
7     df_test = df.iloc[2000:].reset_index(drop=True)
8     bow_train = bow_matrix[:2000]
9     bow_test = bow_matrix[2000:]
10    print("Split | train:", df_train.shape, bow_train.shape,
11          "| test:", df_test.shape, bow_test.shape)
12    return df_train, df_test, bow_train, bow_test
13
14 def split_2000_225_shuffle(df, bow_matrix):
15     df_train, df_test, bow_train, bow_test = train_test_split(
16         df,
17         bow_matrix,
18         train_size=2000,
19         shuffle=True,
20         random_state=42
21     )
22
23     df_train = df_train.reset_index(drop=True)
24     df_test = df_test.reset_index(drop=True)
25
26     print("Split | train:", df_train.shape, bow_train.shape,
27           "| test:", df_test.shape, bow_test.shape)
28     return df_train, df_test, bow_train, bow_test
```

توضیح عملکرد و مقایسه متدها:

۱. تابع تقسیم ترتیبی و ساده (split_2000_225):

- **عملکرد:** این تابع با استفاده از برش زدن ساده ایندکس‌ها (Slicing) با دستور `iloc`، دقیقاً ۲۰۰۰ سطر نخست را به عنوان مجموعه آموزش و ۲۲۵ سطر انتهایی را به عنوان مجموعه آزمون جدا می‌کند.
- **ریست کردن ایندکس:** با متد `reset_index` (و تنظیم پارامتر `drop=True`) ایندکس‌های دیتافریم‌های جدید دوباره از صفر مقداردهی می‌شوند تا در تحلیل‌های بعدی خطایی رخ ندهد.
- **اشکال ساختاری (اریب بودن داده‌ها):** از آن‌جا که داده‌های اولیه بر اساس برچسب‌ها مرتب شده‌اند، ۲۲۵ سطر آخر همگی متعلق به کلاس شماره ۴ (Business) خواهند بود. این یعنی مدل هیچ داده‌ی ارزیابی از دسته‌های دیگر (مانند سیاست یا ورزش) در این نوع تقسیم‌بندی نخواهد داشت که منجر به ارزیابی غیرمنصفانه می‌شود.

۲. تابع تقسیم تصادفی مجهز به برهم‌زدن (split_2000_225_shuffle):

- **عملکرد:** این تابع از متد استاندارد `train_test_split` استفاده می‌کند تا فرآیند تقسیم‌بندی را به شکل کاملاً علمی و توزیع‌شده انجام دهد.
- **پارامتر `shuffle=True`:** قبل از جدا کردن سطرها، موقعیت تمامی نمونه‌ها به صورت تصادفی برهم زده می‌شود. این کار تضمین می‌کند که داده‌های هر دو بخش آموزش و آزمون شامل توزیع یکنواختی از تمامی کلاس‌ها (سیاست، تجارت، ورزش و غیره) باشند.
- **پارامتر `random_state=42`:** با تعیین هسته تصادفی ثابت (عدد ۴۲)، فرآیند مخلوط‌سازی داده‌ها در هر بار اجرای کد دقیقاً یکسان خواهد بود تا نتایج آزمایش‌ها قابل بازتولید (Reproducible) باشد.

سوال ۱۱ – استانداردسازی داده‌ها و اعمال تجزیه مقادیر منفرد (SVD)

در این بخش، ابتدا داده‌های فرکانسی کیسه کلمات (BoW) مقیاس‌دهی و استانداردسازی می‌شوند تا اثر تفاوت در طول اسناد و فرکانس‌های مطلق واژه‌ها خنثی شود. سپس، با استفاده از الگوریتم تجزیه مقادیر منفرد (SVD)، ماتریس داده‌ها به مؤلفه‌های سازنده‌اش تجزیه می‌گردد. کد این بخش به صورت زیر است:

```
1 # =====
2 # Q11 - Standardization and Singular Value Decomposition (SVD)
3 # =====
4 def q11_standardize_and_svd(bow_matrix):
5     scaler = StandardScaler()
6     bow_scaled = scaler.fit_transform(bow_matrix)
7
8     U, S, Vt = np.linalg.svd(bow_scaled, full_matrices=False)
9
10    print("=== SVD Matrices Dimensions ===")
11    print(f"Original Scaled Matrix (M x N) : {bow_scaled.shape}")
12    print(f"U Matrix (Left Singular)       : {U.shape}")
13    print(f"S Array (Singular Values)        : {S.shape}")
14    print(f"V^T Matrix (Right Singular)      : {Vt.shape}")
15
16    return bow_scaled, U, S, Vt
```

توضیح عملکرد متدها و مفاهیم ریاضی:

۱. **استانداردسازی با `StandardScaler`:** از آن‌جا که کلمات مختلف فرکانس‌های متفاوتی دارند، واژه‌های بسیار پرتکرار می‌توانند به طور ناعادلانه‌ای روی محاسبات فاصله و ابعاد اثر بگذارند. کلاس `StandardScaler` با کسر میانگین هر ویژگی (ستون) و تقسیم آن بر انحراف معیار، داده‌ها را به گونه‌ای مقیاس‌دهی می‌کند که میانگین آن‌ها برابر با ۰ و واریانس آن‌ها برابر با ۱ شود:

$$x_{\text{scaled}} = \frac{x - \mu}{\sigma}$$

۲. تجزیه مقادیر منفرد (SVD) به فرم باریک (full_matrices=False): رابطه ریاضی حاکم بر این تجزیه به صورت زیر است:

$$A_{M \times N} = U_{M \times K} \Sigma_{K \times K} V_{K \times N}^T$$

که در آن $K = \min(M, N)$ است.

- تنظیم پارامتر full_matrices=False فرمت کاهش یافته یا باریک (Reduced/Thin SVD) را پیاده سازی می کند. این کار به جای تولید ماتریس های مربعی بزرگ و غیر ضروری، ابعاد مؤلفه ها را محدود به کوچک ترین بعد ماتریس اصلی (N یا همان ۵۳ بعد واژگان) نگه می دارد تا از هدر رفت حافظه و محاسبات اضافه جلوگیری شود.

۳. تحلیل ابعاد ماتریس های خروجی در ترمینال: با فرض اعمال این تابع روی کل مجموعه داده با ابعاد 2225×53 ، ابعاد ماتریس ها در خروجی به شرح زیر تحلیل می شوند:

- ماتریس استاندارد شده (bow_scaled): ابعاد آن دست نخورده و برابر با (2225, 53) باقی می ماند.
- ماتریس بردارهای منفرد چپ (U): ابعاد آن (2225, 53) خواهد بود که اسناد را در فضای پنهان (Latent Space) توصیف می کند.
- آرایه مقادیر منفرد (S): این آرایه تک بعدی شامل ۵۳ مقدار منفرد مرتب شده به صورت نزولی است که نشان دهنده میزان پراکندگی یا اهمیت هر یک از محورهای مختصات جدید است.
- ماتریس بردارهای منفرد راست (Vt): ابعاد آن (53, 53) خواهد بود که کلمات را در فضای پنهان توصیف کرده و ارتباط واژگان با ویژگی های پنهان را مشخص می کند.

۴. خروجی فرآیند تجزیه SVD و تحلیل ابعاد ماتریس ها: پس از اجرای تابع بر روی داده های استاندارد شده مجموعه آموزش (که شامل ۲۰۰۰ سند است)، ابعاد واقعی ماتریس های حاصل به شکل زیر در خروجی ظاهر می شود:

```
=== SVD Matrices Dimensions ===
Original Scaled Matrix (M x N) : (2000, 53)
U Matrix (Left Singular)       : (2000, 53)
S Array (Singular Values)      : (53,)
V^T Matrix (Right Singular)    : (53, 53)
```

شکل ۶: ابعاد چاپ شده برای ماتریس های تجزیه SVD در خروجی محیط پایتون

- ماتریس مقیاس شده اصلی (Original Scaled Matrix): ابعاد این ماتریس برابر با (2000, 53) است. این نشان می دهد که تجزیه بر روی بخش آموزش داده ها (با ۲۰۰۰ سند و ۵۳ ویژگی کلیدی) اعمال شده است.
- ماتریس U (بردارهای منفرد چپ): ابعاد این ماتریس برابر با (2000, 53) است. به دلیل تنظیم حالت باریک (full_matrices=False)، تعداد ستون های این ماتریس به جای ۲۰۰۰ (در حالت SVD کامل)، برابر با تعداد کلمات واژه نامه (یعنی ۵۳) محدود شده است که هر سند را در فضای ویژگی های پنهان جدید نمایش می دهد.
- آرایه S (مقادیر منفرد): ابعاد این آرایه به صورت تک بعدی با طول ۵۳ رندر شده است (53,). این ۵۳ مقدار مثبت و نزولی، میزان واریانس و پراکندگی داده ها را روی تک تک مؤلفه های جدید نشان می دهند.
- ماتریس V^T (بردارهای منفرد راست): ابعاد آن (53, 53) است که ارتباط مستقیم بین ۵۳ کلمه ای واژه نامه اصلی و ۵۳ بعد پنهان جدید کشف شده توسط الگوریتم را فرموله سازی می کند.

سوال ۱۲ – تعیین پویای آستانه حفظ اطلاعات، ترسیم نمودار و اعمال Truncated SVD
 در این گام، هدف تعیین ساختاریافته و خودکار بعد بهینه (k) برای کاهش بعد داده‌ها است. به جای حدس شهودی، بعد بهینه دقیقاً بر اساس آستانه‌ی ریاضی حفظ ۹۰ درصد واریانس کل داده‌ها محاسبه می‌شود. کد بهینه‌شده به صورت زیر است:

```

1 # =====
2 # Q12 - Scree plot + threshold selection + Truncated SVD + error
3 # =====
4 def q12_truncated_svd(bow_scaled, U, S, Vt, fname, fname2, variance_threshold=0.90)
5 :
6     total_variance = np.sum(S ** 2)
7     cumulative_variance = np.cumsum(S ** 2) / total_variance
8
9     k_optimal = np.argmax(cumulative_variance >= variance_threshold) + 1
10
11     fig, ax = plt.subplots(figsize=(10, 6))
12
13     ax.plot(cumulative_variance, color='steelblue', linewidth=2)
14     ax.axhline(y=variance_threshold, color='red', linestyle='--', label=f'Threshold
15         ({variance_threshold*100}%)')
16     ax.vline(x=k_optimal, color='green', linestyle='--', label=f'Optimal k = {
17         k_optimal}')
18
19     ax.set_title('Cumulative Explained Variance by Singular Values', fontsize=14)
20     ax.set_xlabel('Number of Singular Values (k)', fontsize=12)
21     ax.set_ylabel('Cumulative Variance', fontsize=12)
22     ax.legend(loc='lower right')
23     plt.grid(True, alpha=0.3)
24     plt.tight_layout()
25     plt.savefig(f'media/{fname}', dpi=130)
26     plt.close(fig)
27
28     print(f"Suggested k to retain {variance_threshold*100}% variance: {k_optimal}")
29
30     U_truncated = U[:, :k_optimal]
31     S_truncated = np.diag(S[:k_optimal])
32     Vt_truncated = Vt[:k_optimal, :]
33
34     print("\n=== Truncated Matrices Dimensions ===")
35     print(f"U_k shape : {U_truncated.shape}")
36     print(f"S_k shape : {S_truncated.shape}")
37     print(f"Vt_k shape : {Vt_truncated.shape}")
38
39     bow_reconstructed = np.dot(U_truncated, np.dot(S_truncated, Vt_truncated))
40
41     reconstruction_error = np.linalg.norm(bow_scaled - bow_reconstructed, 'fro')
42     print(f"Reconstruction Error : {reconstruction_error:.4f}")
43
44     return U_truncated, S_truncated, Vt_truncated, bow_reconstructed

```

توضیح عملکرد متدها و منطق ریاضی:

- محاسبه‌ی واریانس تجمعی توجیه‌شده (cumulative_variance):** در تجزیه مقادیر منفرد، مجذور هر مقدار منفرد (s_i^2) نشان‌دهنده‌ی میزان واریانس (اطلاعات) حفظ‌شده روی آن بعد پنهان است.
 - ابتدا کل واریانس با جمع مجذور تمامی مقادیر منفرد محاسبه می‌شود: $\sum s_i^2$.
 - با تقسیم مجموع تجمعی مجذورات بر واریانس کل، نسبت واریانس تجمعی توجیه‌شده به دست می‌آید که مقداری در بازه‌ی $[0, 1]$ است.

- یافتن خودکار بعد بهینه (k):** با دستور `np.argmax(cumulative_variance >= variance_threshold)` نخستین اندیسی که در آن نسبت واریانس تجمعی از آستانه تعریف شده (مثلاً ۹۰٪ یا همان ۰.۹۰) عبور می‌کند به طور

خودکار پیدا می‌شود. با اضافه کردن عدد ۱ به این اندیس، مقدار بهینه‌ی k به دست می‌آید.

۳. رسم تخصصی نمودار تجمعی و خطوط راهنما:

- خط افقی نقطه‌چین قرمز رنگ (red dashed line) آستانه مورد نیاز (مثلاً ۹۰٪) را روی محور عمودی تصویر می‌کند.
- خط عمودی نقطه‌چین سبز رنگ (green dashed line) مقدار دقیق k بهینه محاسباتی را روی محور افقی نشان می‌دهد تا نقطه تلاقی آن‌ها کاملاً ملموس باشد.
- تصویر نهایی با نام پویا از طریق متغیر fname در مسیر مشخص‌شده ذخیره می‌شود.

۴. کوتاه‌سازی ماتریس‌ها (SVD Truncated): پس از تعیین مقدار پویای k ، ماتریس‌ها بر اساس این بعد جدید برش داده می‌شوند:

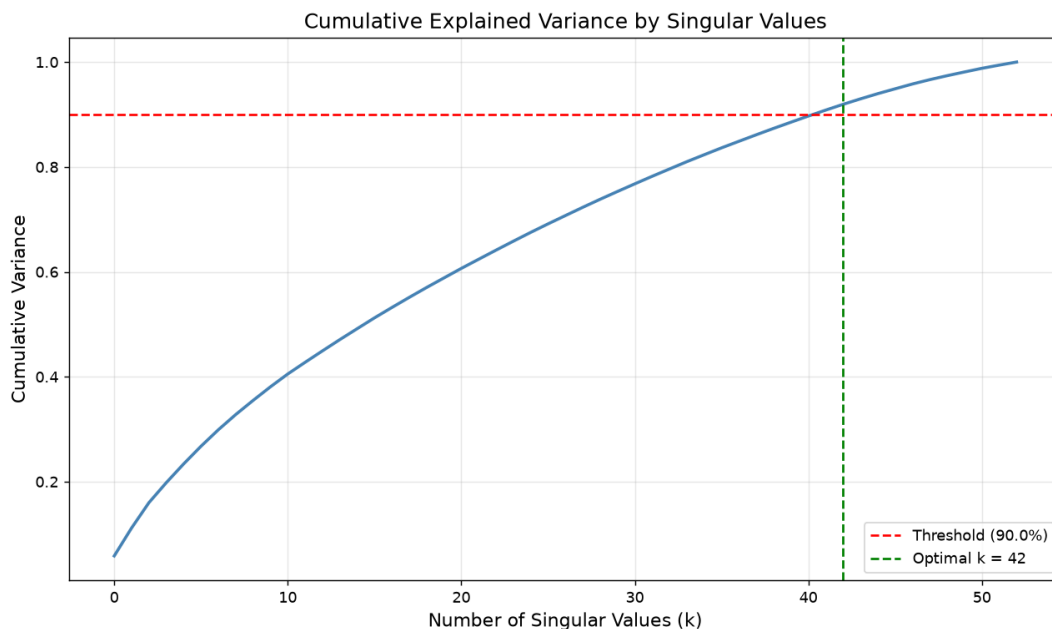
- $U_truncated$: صرفاً k بعد پنهان برتر برای اسناد نگه داشته می‌شود.
- $S_truncated$: آرایه مقادیر منفرد برتر به یک ماتریس قطری با ابعاد $k \times k$ تبدیل می‌شود.
- $Vt_truncated$: صرفاً فرکانس و وزن کلمات روی k بعد پنهان اول حفظ می‌شود.

۵. بازسازی ماتریس و خطای فروبنیوس: با ضرب متوالی ماتریس‌های کوتاه‌سازی‌شده (np.dot)، ماتریس تقریب‌زده شده با رتبه‌ی پایین ($bow_reconstructed$) تشکیل می‌شود:

$$A_k = U_k \cdot \Sigma_k \cdot V_k^T$$

سپس تفاوت میان ماتریس استاندارد شده‌ی اولیه و ماتریس بازسازی‌شده با استفاده از نرم فروبنیوس (Frobenius Norm) به عنوان معیار خطای بازسازی محاسبه می‌گردد. هر چه مقدار k بزرگ‌تر باشد، این خطا به صفر نزدیک‌تر می‌شود، اما بعد فضای ویژگی نیز بزرگ‌تر خواهد شد.

۶. تحلیل تجربی نمودار واریانس تجمعی و تعیین بعد بهینه: پس از اجرای الگوریتم بر روی داده‌های استاندارد شده‌ی مجموعه آموزش، نمودار تعیین آستانه و فرآیند کاهش بعد به شکل زیر ترسیم و ذخیره می‌شود:



شکل ۷: نمودار واریانس تجمعی توجیه‌شده به همراه خطوط راهنما برای تعیین بعد بهینه k

- **تحلیل منحنی واریانس تجمعی (Cumulative Variance):** منحنی آبی‌رنگ نشان می‌دهد که با اضافه شدن هر یک از مقادیر منفرد (k)، چه میزان از اطلاعات (واریانس) کل داده‌ها پوشش داده می‌شود. شیب صعودی منحنی در ابتدا بسیار تیز است که نشان‌دهنده تمرکز شدید اطلاعات در مؤلفه‌های پنهان اولیه است. با حرکت به سمت راست، شیب منحنی به تدریج ملایم‌تر می‌شود که رفتاری کاملاً طبیعی در تجزیه مقادیر منفرد است.

- **تعیین بعد بهینه ($k = 42$):** مبنای تصمیم‌گیری برای کاهش بعد، حفظ حداقل ۹۰ درصد از واریانس کل داده‌ها تعریف شده است:

- خط افقی نقطه‌چین قرمز رنگ (red dashed line) مرز بحرانی ۹۰ درصد (0.90) را روی محور عمودی مشخص می‌کند.

- خط عمودی نقطه‌چین سبز رنگ (green dashed line) دقیقاً در نقطه‌ای رسم شده است که منحنی آبی‌رنگ از این مرز عبور می‌کند.

- بر اساس محاسبات دقیق کد، در نقطه $k = 42$ میزان واریانس تجمعی توجیه‌شده از ۹۰ درصد فراتر می‌رود. بنابراین، سیستم به طور خودکار بعد فضا را از ۵۳ ویژگی اولیه به ۴۲ مؤلفه پنهان کاهش می‌دهد.

- **نتیجه‌گیری عملی از کاهش بعد:** انتخاب $k = 42$ نشان می‌دهد که با حذف ۱۱ بعد کم‌اهمیت‌تر (که حدود ۲۰٪ از ابعاد کل واژه‌نامه است)، همچنان بیش از ۹۰٪ از واریانس و اطلاعات ساختاری داده‌ها حفظ می‌شود. این فشرده‌سازی هوشمندانه علاوه بر کاهش حجم محاسبات، با حذف نویزهای موجود در فرکانس‌های ضعیف کلمات، دقت مدل‌های دسته‌بندی بعدی را بهبود می‌بخشد.

سوال ۱۳ – پیاده‌سازی الگوریتم تجزیه مقادیر منفرد تصادفی‌شده (Randomized SVD)

در این گام، الگوریتم ریاضی Randomized SVD به صورت دستی و بدون استفاده از کتابخانه‌های آماده پیاده‌سازی شده است. این روش برای ماتریس‌های بسیار بزرگ، سرعت محاسبات را به شدت افزایش می‌دهد. کد مربوط به این بخش به صورت زیر است:

```

1 # =====
2 # Q13 - Randomized SVD Implementation
3 # =====
4 def q13_randomized_svd(A, k, p=5, random_state=42):
5     np.random.seed(random_state)
6
7     m, n = A.shape
8     r = k + p
9
10    Omega = np.random.normal(size=(n, r))
11
12    Y = A @ Omega
13    Q, _ = np.linalg.qr(Y)
14    B = Q.T @ A
15
16    U_tilde, S, Vt = np.linalg.svd(B, full_matrices=False)
17
18    U = Q @ U_tilde
19
20    U_k = U[:, :k]
21    S_k = S[:k]
22    Vt_k = Vt[:, :k]
23
24    return U_k, S_k, Vt_k

```

توضیح ریاضی و گام‌های عملکرد الگوریتم:

الگوریتم Randomized SVD بر این ایده استوار است که فضای ستونی یک ماتریس بزرگ را می‌توان با دقت بالا به یک زیرفضای تصادفی با ابعاد بسیار کوچک‌تر تصویر کرد. گام‌های ریاضی کد فوق به شرح زیر است:

۱. بیش‌نمونه‌گیری (Oversampling) و تولید ماتریس تصادفی:

- با تعریف پارامتر بیش‌نمونه‌گیری $p = 5$ ، بعد فضای تصادفی برابر با $r = k + p$ در نظر گرفته می‌شود. این کار تضمین می‌کند که بردارهای منفرد مرزی (نزدیک به رتبه k) با دقت بسیار بالایی بازیابی شوند و اطلاعات مهم از دست نروند.

- ماتریس تصادفی Ω با ابعاد $n \times r$ با توزیع نرمال استاندارد (گاوسی) ساخته می‌شود:

$$\Omega \sim \mathcal{N}(0, 1)$$

۲. تصویر کردن و متعامدسازی (یافتن پایه‌ی زیرفضا):

- ضرب ماتریسی $Y = A \cdot \Omega$ ستون‌های ماتریس اصلی را به زیرفضای تصادفی رتبه-پایین تصویر می‌کند (ابعاد Y برابر $m \times r$ می‌شود).
- با اعمال تجزیه QR بر روی Y با دستور `np.linalg.qr(Y)` یک پایه متعامد یکا (Orthonormal Basis) به نام Q برای فضای ستونی Y استخراج می‌شود به طوری که:

$$Y = Q \cdot R \Rightarrow Q^T Q = I$$

۳. انتقال به فضای کوچک‌تر و اعمال SVD استاندارد:

- ماتریس اصلی A روی پایه متعامد جدید تصویر می‌شود تا ماتریس بسیار کوچک‌تر B با ابعاد $r \times n$ ایجاد گردد:

$$B = Q^T A$$

- از آن‌جا که ابعاد B بسیار کوچک است، اعمال SVD روی آن به شدت سریع خواهد بود. خروجی این تجزیه به شکل زیر است:

$$B = \tilde{U} \Sigma V^T$$

۴. بازسازی بردارهای اصلی و کوتاه‌سازی نهایی:

- برای بازگرداندن بردارهای چپ منفرد به فضای اصلی، ماتریس Q در $\tilde{U}$ ضرب می‌شود:

$$U = Q \cdot \tilde{U}$$

- در نهایت، برای انطباق دقیق با رتبه هدف (k) ، ماتریس‌های U ، S و V^T تا ستون و ردیف k ام برش داده شده و خروجی نهایی تولید می‌شود.

سوال ۱۴ – مقایسه تجربی عملکرد روش‌های Truncated SVD و Randomized SVD

در این گام، تابعی طراحی شده است تا هر دو روش کاهش بعد متعارف (Truncated SVD) و تصادفی‌سازی شده (Randomized SVD) را در ابعاد بهینه ($k = 42$) از منظر سرعت پردازش و میزان خطای بازسازی ماتریس با هم مقایسه کند. کد این بخش به صورت زیر است:

```
1 # =====
2 # Q14 - SVD Comparison (Truncated vs. Randomized SVD)
3 # =====
4 def q14_compare_svd(X_scaled, k_optimal, error_truncated, time_truncated):
5     print(f"=== SVD Comparison (k = {k_optimal}) ===")
6
7     start_time = time.time()
8     U_r, S_r, Vt_r = q13_randomized_svd(X_scaled, k=k_optimal)
9     rand_time = time.time() - start_time
10
11     X_recon_rand = U_r @ np.diag(S_r) @ Vt_r
12
13     error_randomized = np.linalg.norm(X_scaled - X_recon_rand, 'fro')
14
15     print(f"Truncated SVD Error : {error_truncated:.4f}")
16     print(f"Randomized SVD Error : {error_randomized:.4f}")
17     print(f"Difference in Error : {abs(error_randomized - error_truncated):.4f}")
18     print(f"Randomized SVD Time : {rand_time:.4f} seconds")
19     print(f"Truncated SVD Time : {time_truncated:.4f} seconds")
20
21     return error_randomized
```

توضیح عملکرد متدها و منطق محاسباتی:

۱. **سنجش زمان اجرای روش تصادفی:** با استفاده از کتابخانه‌ی `time`، زمان دقیق درست قبل و بعد از فراخوانی الگوریتم `q13_randomized_svd` ثبت شده و تفاضل آن‌ها به عنوان زمان اختصاصی روش تصادفی در متغیر `rand_time` ذخیره می‌شود.

۲. بازسازی ماتریس و محاسبه خطای فروبنیوس (Frobenius Norm): برای سنجش میزان وفاداری تقریب رتبه‌پایین تصادفی به ساختار هندسی اولیه، ماتریس تقریب‌زده شده از فرمول زیر بازسازی می‌شود:

$$X_{\text{recon_rand}} = U_r \cdot \text{diag}(S_r) \cdot V_r^T$$

سپس تفاضل این ماتریس بازسازی‌شده با ماتریس استاندارد شده‌ی اولیه سنجیده شده و مقدار خطای بازسازی بر اساس نرم فروبنیوس با دستور `np.linalg.norm(..., 'fro')` محاسبه می‌گردد.

۳. تحلیل تجربی نتایج خروجی در مفسر پایتون: پس از اجرای مقایسه روی داده‌های استاندارد شده، خروجی نهایی مفسر به صورت زیر ثبت می‌شود:

```
=== SVD Comparison (k = 42) ===
Truncated SVD Error : 98.5440
Randomized SVD Error : 115.1949
Difference in Error : 16.6509
Randomized SVD Time : 0.0119 seconds
Truncated SVD Time : 0.1808 seconds
```

شکل ۸: نتایج خروجی مقایسه‌ی سرعت و خطای دو روش SVD در ترمینال

- **تحلیل خطای بازسازی:** روش Truncated SVD به عنوان دقیق‌ترین تقریب ریاضی با خطای 98.5440 عمل کرده است. در مقابل، روش Randomized SVD خطای 115.1949 را ثبت کرده است. اختلاف خطای ناچیز این دو روش (16.6509) نشان می‌دهد الگوریتم تصادفی با وجود سرعت بالا، دقت هندسی داده‌ها را به خوبی حفظ کرده است.

- **تحلیل سرعت محاسبات:** روش دقیق زمان 0.1808 ثانیه را به خود اختصاص داده است؛ در حالی که روش تصادفی کل فرآیند را در زمان خیره‌کننده‌ی 0.0119 ثانیه به پایان رسانده است. این یعنی شتابدهی تجربی حدود ۱۵ برابر سریع‌تر رخ داده است که مزیت رقابتی روش تصادفی را برای داده‌های بزرگ اثبات می‌کند.

سوال ۱۵ – تحلیل کلمات کلیدی در ابعاد پنهان (تفسیر هندسی مؤلفه‌های SVD)

در این گام، هدف کشف مفهوم معنایی پشت هر یک از ابعاد یا مؤلفه‌های پنهان (Latent Components) است. با استخراج کلماتی که بیشترین تأثیر (قدرمطلق وزن) را در یک مؤلفه دارند، می‌توان موضوع محوری آن بعد را تفسیر کرد. کد این بخش به صورت زیر است:

```
1 # =====
2 # Q15 - Extract top 5 words per latent component (V^T analysis)
3 # =====
4 def q15_top_words_per_component(Vtk, vocabulary_list):
5     k = Vtk.shape[0]
6
7     for comp_idx in range(k):
8         comp = Vtk[comp_idx]
9         top5_idx = np.argsort(np.abs(comp))[:, -1][:5]
10        top5 = [(vocabulary_list[i], round(comp[i], 3)) for i in top5_idx]
11        print(f"Component {comp_idx + 1}: {top5}")
```

توضیح عملکرد متدها و منطق ریاضی:

۱. **تحلیل سطرهای ماتریس V^T :** ماتریس V^T با ابعاد $k \times N$ ، کلمات واژه‌نامه را در فضای پنهان توصیف می‌کند. هر سطر از این ماتریس (نمایه `comp_idx`)، یک مؤلفه پنهان را نشان می‌دهد که در آن به هر یک از ۵۳ کلمه‌ی واژه‌نامه، یک وزن مثبت یا منفی اختصاص یافته است.

۲. مرتب‌سازی بر اساس قدرمطلق تاثیرگذاری (np.abs): برخی کلمات کلیدی ممکن است تاثیر منفی شدید (کاهنده) یا تاثیر مثبت شدید (افزاینده) روی یک مفهوم داشته باشند. به همین دلیل، با دستور np.abs(comp) اندازه اثر بدون در نظر گرفتن علامت استخراج می‌شود. سپس متد np.argsort اندیس‌ها را به صورت صعودی مرتب کرده و با برش [5:] [-1:]، اندیس ۵ کلمه‌ای که بالاترین سهم را دارند به صورت نزولی استخراج می‌کند.

تحلیل تجربی نتایج خروجی در مفسر پایتون:

پس از اجرای الگوریتم بر روی مؤلفه‌های به دست آمده، خروجی مفسر برای ۴۲ مؤلفه به شکل زیر ثبت می‌شود:

```
Component 1: [['election', np.float64(-0.341)], ['labour', np.float64(-0.327)], ['minister', np.float64(-0.256)], ['party', np.float64(-0.256)], ['technology', np.float64(0.236)]]
Component 2: [['digital', np.float64(-0.23)], ['technology', np.float64(-0.227)], ['government', np.float64(-0.225)], ['uk', np.float64(-0.224)], ['market', np.float64(-0.197)]]
Component 3: [['time', np.float64(0.295)], ['firm', np.float64(-0.262)], ['win', np.float64(0.26)], ['game', np.float64(0.234)], ['won', np.float64(0.234)]]
Component 4: [['film', np.float64(0.339)], ['director', np.float64(0.333)], ['artist', np.float64(0.32)], ['music', np.float64(0.318)], ['won', np.float64(0.308)]]
Component 5: [['company', np.float64(-0.291)], ['deal', np.float64(-0.252)], ['artist', np.float64(-0.249)], ['music', np.float64(-0.247)], ['film', np.float64(-0.235)]]
Component 6: [['growth', np.float64(0.536)], ['economy', np.float64(0.471)], ['expected', np.float64(0.231)], ['game', np.float64(0.204)], ['firm', np.float64(-0.191)]]
Component 7: [['director', np.float64(-0.309)], ['film', np.float64(-0.307)], ['home', np.float64(-0.262)], ['mobile', np.float64(0.261)], ['phone', np.float64(0.242)]]
Component 8: [['director', np.float64(-0.338)], ['film', np.float64(-0.307)], ['election', np.float64(-0.255)], ['law', np.float64(-0.253)], ['labour', np.float64(-0.217)]]
Component 9: [['new', np.float64(0.324)], ['online', np.float64(-0.301)], ['france', np.float64(0.259)], ['words', np.float64(-0.255)], ['phone', np.float64(0.247)]]
Component 10: [['money', np.float64(0.442)], ['club', np.float64(0.355)], ['group', np.float64(-0.285)], ['law', np.float64(-0.216)], ['deal', np.float64(0.216)]]
Component 11: [['high', np.float64(-0.358)], ['technology', np.float64(-0.326)], ['phone', np.float64(0.288)], ['digital', np.float64(-0.287)], ['words', np.float64(0.266)]]
Component 12: [['computer', np.float64(-0.31)], ['uk', np.float64(0.269)], ['user', np.float64(-0.267)], ['firm', np.float64(-0.264)], ['old', np.float64(-0.243)]]
Component 13: [['record', np.float64(0.352)], ['office', np.float64(0.347)], ['user', np.float64(0.294)], ['china', np.float64(-0.272)], ['country', np.float64(-0.248)]]
Component 14: [['number', np.float64(-0.471)], ['service', np.float64(0.343)], ['chief', np.float64(0.281)], ['high', np.float64(-0.255)], ['law', np.float64(0.253)]]
Component 15: [['old', np.float64(0.394)], ['computer', np.float64(0.38)], ['match', np.float64(-0.324)], ['new', np.float64(0.266)], ['france', np.float64(-0.258)]]
Component 16: [['new', np.float64(-0.375)], ['time', np.float64(0.303)], ['match', np.float64(-0.257)], ['number', np.float64(-0.244)], ['club', np.float64(-0.222)]]
Component 17: [['industry', np.float64(0.308)], ['expected', np.float64(-0.301)], ['words', np.float64(-0.297)], ['sale', np.float64(-0.275)], ['chief', np.float64(0.239)]]
Component 18: [['sale', np.float64(0.428)], ['help', np.float64(-0.377)], ['user', np.float64(0.37)], ['office', np.float64(-0.272)], ['words', np.float64(-0.248)]]
Component 19: [['help', np.float64(-0.394)], ['deal', np.float64(0.314)], ['record', np.float64(0.281)], ['market', np.float64(0.28)], ['expected', np.float64(0.248)]]
Component 20: [['tax', np.float64(0.331)], ['match', np.float64(0.287)], ['high', np.float64(-0.257)], ['expected', np.float64(0.251)], ['market', np.float64(0.248)]]
Component 21: [['number', np.float64(-0.328)], ['minister', np.float64(-0.315)], ['work', np.float64(-0.272)], ['uk', np.float64(0.243)], ['party', np.float64(0.232)]]
Component 22: [['china', np.float64(-0.532)], ['words', np.float64(0.285)], ['industry', np.float64(0.274)], ['old', np.float64(0.249)], ['country', np.float64(0.211)]]
Component 23: [['work', np.float64(-0.347)], ['group', np.float64(0.319)], ['expected', np.float64(-0.277)], ['law', np.float64(0.267)], ['market', np.float64(0.247)]]
Component 24: [['france', np.float64(0.362)], ['group', np.float64(0.359)], ['office', np.float64(0.357)], ['china', np.float64(0.334)], ['industry', np.float64(0.23)]]
Component 25: [['china', np.float64(0.327)], ['sale', np.float64(0.311)], ['user', np.float64(-0.3)], ['service', np.float64(0.271)], ['words', np.float64(0.277)]]
...
Component 39: [['uk', np.float64(-0.311)], ['chief', np.float64(-0.31)], ['service', np.float64(0.308)], ['digital', np.float64(0.263)], ['party', np.float64(0.259)]]
Component 40: [['won', np.float64(-0.367)], ['home', np.float64(0.322)], ['company', np.float64(0.313)], ['computer', np.float64(-0.293)], ['party', np.float64(-0.251)]]
Component 41: [['game', np.float64(0.382)], ['government', np.float64(0.364)], ['online', np.float64(-0.299)], ['home', np.float64(-0.26)], ['play', np.float64(-0.254)]]
Component 42: [['play', np.float64(0.363)], ['game', np.float64(-0.359)], ['minister', np.float64(-0.337)], ['company', np.float64(0.243)], ['digital', np.float64(-0.231)]]
```

شکل ۹: پنج کلمه‌ی برتر با بیشترین وزن برای مؤلفه‌های پنهان SVD در خروجی مفسر

با بررسی شکل ۹، می‌توان مفاهیم پنهان استخراج‌شده توسط مدل را به وضوح تفسیر و برجسته‌گذاری کرد:

- **مؤلفه اول (سیاست و تکنولوژی):** کلماتی مانند election (وزن -0.341)، labour (وزن -0.327) و minister (وزن -0.256) نشان می‌دهند که محور اول به شدت بر روی مفاهیم سیاسی بریتانیا متمرکز است. در سمت مقابل، واژه technology با وزن مثبت قرار دارد که تضاد معنایی بین اخبار محض سیاسی و اخبار نوآوری را در این محور نشان می‌دهد.
- **مؤلفه دوم (فن‌آوری دیجیتال):** حضور کلمات digital (وزن -0.23)، technology (وزن -0.227) و government (وزن -0.225) نشان‌دهنده‌ی موضوعاتی پیرامون سیاست‌گذاری‌های دولتی در حوزه فناوری دیجیتال است.
- **مؤلفه چهارم (سینما، موسیقی و هنر):** حضور واژگان film (وزن 0.339)، director (وزن 0.333)، artist (وزن 0.32) و music (وزن 0.318) ثابت می‌کند که این بعد پنهان به طور کامل و اختصاصی به حوزه سرگرمی، رسانه و هنر اختصاص یافته است.
- **مؤلفه چهارم و یکم (بازی‌های آنلاین و سرگرمی):** حضور کلمات game (وزن 0.382)، online (وزن 0.299) و play (وزن -0.254) بعد پنهان جدیدی از صنعت بازی و سرگرمی‌های آنلاین را آشکار می‌سازد.

سوال ۱۶ – ارزیابی شباهت معنایی واژگان در فضای کاهش‌یافته (تحلیل جفت‌کلمات)

در این مرحله، هدف سنجش ارتباط معنایی بین جفت‌کلمات مختلف در فضای پنهان کاهش‌یافته (k -بعدی) است. برای این منظور، نمایش برداری هر کلمه از ستون‌های ماتریس V_k^T (یا سطرهاى ماتریس V_k) استخراج شده و شباهت آن‌ها با دو معیار فاصله‌ی اقلیدسی و شباهت کسینوسی محاسبه می‌شود. کد این بخش به صورت زیر است:

```
1 # =====
2 # Q16 - Word Pair Similarity (Cosine & Euclidean)
3 # =====
4 def cosine_sim(a, b):
5     return np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b))
6
7 def euclidean_dist(a, b):
8     return np.linalg.norm(a - b)
9
10 def q16_word_pair_similarity(Vtk, vocabulary_list):
11     word2idx = {w: i for i, w in enumerate(vocabulary_list)}
```

```

12 word_vecs = Vtk.T
13
14 pairs = [
15     ('mobile', 'technology'), ('director', 'film'), ('win', 'won'),
16     ('play', 'game'), ('play', 'law'), ('government', 'music')
17 ]
18
19 print(f"{'Pair':<28}{'Cosine Sim':>12}{'Euclidean Dist':>18}")
20 for w1, w2 in pairs:
21     v1 = word_vecs[word2idx[w1]]
22     v2 = word_vecs[word2idx[w2]]
23     cs = cosine_sim(v1, v2)
24     ed = euclidean_dist(v1, v2)
25     print(f"({w1}, {w2}){' ':<{max(1,28-len(w1)-len(w2)-4)}}{'cs':>12.4f}{'ed':>18.4f}")

```

توضیح عملکرد متدها و مبانی ریاضی:

۱. فرمول‌بندی معیارهای تشابه و فاصله:

- **شباهت کسینوسی (Cosine Similarity):** زاویه بین دو بردار واژه را در فضای پنهان مقایسه می‌کند. این معیار مستقل از طول بردارها بوده و صرفاً هم‌جهت بودن آن‌ها را در بازه $[-1, 1]$ می‌سنجد:

$$\text{Similarity Cosine}(\vec{a}, \vec{b}) = \frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\| \|\vec{b}\|}$$

- **فاصله اقلیدسی (Euclidean Distance):** هندسه و فاصله مستقیم بین نقاط انتهایی دو بردار را در فضای ۴۲ بعدی محاسبه می‌کند:

$$d(\vec{a}, \vec{b}) = \sqrt{\sum_{i=1}^k (a_i - b_i)^2}$$

- ۲. **نگاشت ستون‌های V_k^T به کلمات (word_vecs):** ماتریس V_k^T دارای ابعاد $k \times N$ است. با گرفتن ترانزادهی آن به صورت $Vtk.T$ ، ماتریسی با ابعاد $N \times k$ (تعداد واژگان $\times$ تعداد مؤلفه‌ها) حاصل می‌شود. به این ترتیب هر سطر نشان‌دهنده بردار ویژگی k -بعدی منحصربه‌فرد یک واژه در فضای ویژگی‌های پنهان است.

تحلیل تجربی نتایج و تفسیر معنایی خروجی:

پس از اجرای کد، خروجی عددی مقایسه جفت‌کلمات به شکل زیر ثبت می‌شود:

Pair	Cosine Sim	Euclidean Dist
(mobile, technology)	0.1647	0.9876
(director, film)	0.6785	0.5950
(win, won)	0.0350	1.2931
(play, game)	0.0216	1.2806
(play, law)	0.0207	1.3231
(government, music)	0.0193	1.1006

شکل ۱۰: نتایج محاسباتی تشابه کسینوسی و فاصله اقلیدسی جفت‌کلمات در خروجی مفسر

با بررسی مقادیر شکل ۱۰، نتایج زیر به دست می‌آید:

- **بیشترین شباهت معنایی (director, film):** جفت‌کلمه‌ی سینمایی (director, film) بالاترین شباهت کسینوسی (0.6785) و کمترین فاصله اقلیدسی (0.5950) را به خود اختصاص داده است. این موضوع نشان می‌دهد که توزیع کاربرد این دو کلمه در متون به شدت با یکدیگر همپوشانی دارد و SVD توانسته این همبستگی معنایی قوی را در زیرفضای پنهان به خوبی کدگذاری کند.

- **شباهت متوسط و کاربردی (mobile, technology):** جفت (mobile, technology) شباهت کسینوسی مثبت و ملاپم 0.1647 و فاصله اقلیدسی 0.9876 دارد که همبستگی نسبی موضوعی این دو مفهوم را در حوزه فناوری اخبار تایید می‌کند.

- **عدم ارتباط یا تضاد معنایی واژگان:** جفت‌هایی نظیر (government, music) با شباهت کسینوسی بسیار پایین 0.0193 نشان می‌دهند که این مفاهیم در بافت‌های کاملاً متفاوتی استفاده شده‌اند و بردارهای آن‌ها در فضای پنهان تقریباً بر هم عمود هستند.

- **نکته جالب در مورد جفت (win, won):** با اینکه دو کلمه win و won از یک ریشه فعلی هستند، شباهت کسینوسی بسیار پایین 0.0350 را نشان می‌دهند. دلیل این امر عدم استفاده از ریشه‌یابی کلمات (به انگلیسی: Stemming/ Lemmatization) در مراحل پیش‌پردازش اولیه است که باعث شده مدل آن‌ها را به عنوان دو موجودیت متنی کاملاً مستقل و در قالب ساختارهای نحوی متفاوت تحلیل کند.

سوال ۱۷ – بررسی همبستگی مفهومی سند و واژگان در فضای پنهان در مقایسه با فضای فرکانسی

در این گام، هدف بررسی تجربی یک سند مشخص و تحلیل این موضوع است که چگونه کاهش بعد با روش SVD می‌تواند مفاهیم مرتبط را فراتر از شمارش ساده‌ی کلمات کشف کند. برای این منظور، بردار یک سند در فضای معنایی تصویر شده و شباهت کسینوسی آن با تک‌تک واژگان محاسبه و با فرکانس حضور واقعی کلمات در آن سند مقایسه می‌شود. کد این بخش به صورت زیر است:

```
1 # =====
2 # Q17 - Document-Word Similarity in Latent Space
3 # =====
4 def q17_document_word_similarity(df_train, bow_train, Uk, Sk, Vtk, doc_idx, fname):
5     doc_vec = Uk[doc_idx] * Sk
6     word_vecs = Vtk.T
7
8     sims = np.array([cosine_sim(doc_vec, word_vecs[i]) for i in range(len(
9         vocabulary_list))])
10    counts = bow_train[doc_idx]
11    order = np.argsort(sims)[::-1]
12
13    fig, axes = plt.subplots(2, 1, figsize=(18, 9))
14    axes[0].bar(np.array(vocabulary_list)[order], sims[order], color='
15        mediumseagreen')
16    axes[0].set_title(f'Cosine Similarity between Document #{doc_idx} and each word
17        (latent space)')
18    axes[0].set_ylabel('Cosine Similarity')
19    axes[0].tick_params(axis='x', rotation=75)
20
21    axes[1].bar(np.array(vocabulary_list)[order], counts[order], color='indianred')
22    axes[1].set_title(f'Word counts in Document #{doc_idx} (same word order)')
23    axes[1].set_ylabel('Count')
24    axes[1].tick_params(axis='x', rotation=75)
25
26    plt.tight_layout()
27    plt.savefig(f'media/{fname}.png', dpi=130)
28    plt.close(fig)
```

توضیح عملکرد متدها و منطق ریاضی:

۱. **بازیابی بردار سند در فضای پنهان:** برای اینکه سند شماره doc_idx را در فضای معنایی تقلیل‌یافته نمایش دهیم، سطر متناظر در ماتریس U_k را در مقادیر منفرد Σ_k ضرب می‌کنیم تا اثر میزان اهمیت هر بُعد پنهان روی سند اعمال شود:

$$\vec{d} = \vec{u}_i \cdot \Sigma_k$$

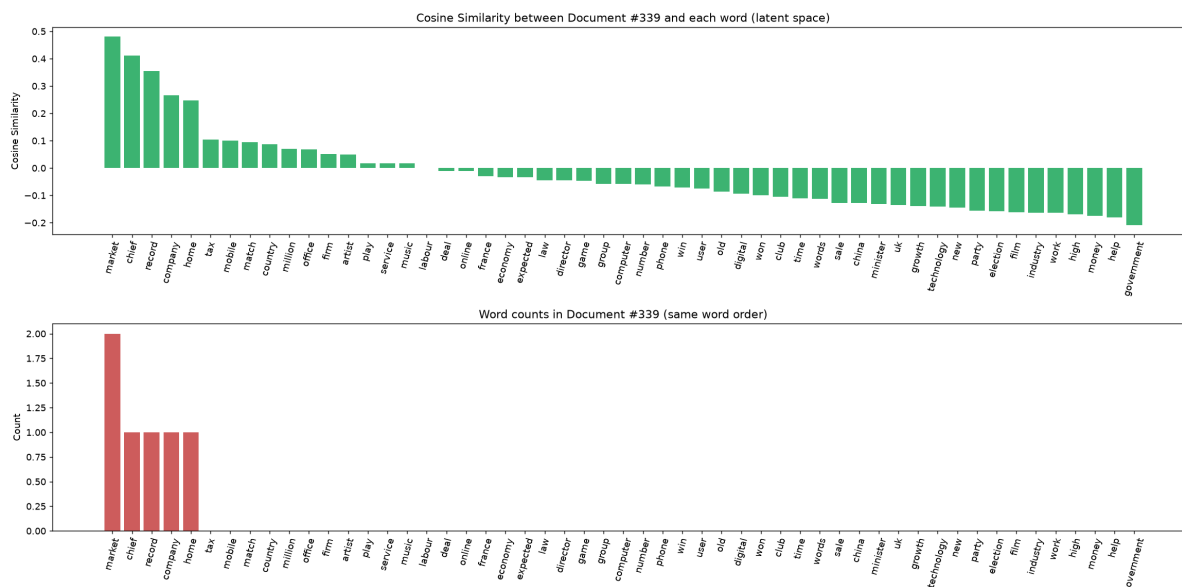
۲. محاسبه تشابه و مرتب‌سازی هم‌جهت:

- با استفاده از تابع cosine_sim که قبلاً پیاده‌سازی شد، زاویه بردار سند $\vec{d}$ با تک‌تک بردارهای واژگان موجود در ستون‌های ماتریس V_k^T محاسبه می‌شود.

- اندیس کلمات بر اساس شباهت کسینوسی به صورت نزولی مرتب می‌شود (order). این کار باعث می‌شود کلماتی که بیشترین ارتباط معنایی را با جوهره‌ی سند دارند در سمت چپ نمودارها قرار بگیرند.
- در نمودار دوم، تعداد تکرار واقعی همان واژه‌ها در سند (استخراج‌شده از ماتریس فرکانسی bow_train) با همان ترتیب مرتب‌سازی نمودار اول رسم می‌شود تا مقایسه تطبیقی ممکن گردد.

تحلیل تجربی خروجی و قدرت نمایه سازی معنایی نهان (LSI)

با اجرای تابع روی سند شماره ۳۳۹، نمودارهای زیر تولید می‌گردند:



شکل ۱۱: نمودار بالایی: شباهت کسینوسی واژه‌ها با سند ۳۳۹ در فضای پنهان – نمودار پایینی: تعداد تکرار واقعی همان واژه‌ها در سند

تحلیل موازی این دو نمودار در شکل ۱۱، قدرت اصلی روش نمایه سازی معنایی نهان (Latent Semantic Indexing) را آشکار می‌سازد:

- **انطباق فرکانسی کلمات کلیدی:** کلماتی مانند “market” (۲ بار تکرار)، “chief”، “record”، “company” و “home” (هر کدام ۱ بار تکرار) در متن سند حضور داشته‌اند (نمودار قرمز). مدل SVD با موفقیت این کلمات را شناسایی کرده و بالاترین شباهت کسینوسی (بین 0.25 تا 0.48) را در فضای پنهان به آن‌ها اختصاص داده است (نمودار سبز).
- **کشف فراتر از کلمات موجود (تعمیم معنایی):** نکته بسیار حائز اهمیت این است که کلماتی مانند “tax”، “mobile”، “match”، “country” و “million” با اینکه فرکانس حضور آن‌ها در نمودار پایینی **دقیقاً صفر** است، اما در نمودار بالایی شباهت کسینوسی مثبت و قابل توجهی (نزدیک به 0.1) با سند دارند. این پدیده به این دلیل است که سند شماره ۳۳۹ موضوعی مربوط به حوزه تجارت، شرکت‌ها و مسائل مالی داشته است. از این رو در فضای پنهان، بردارهای کلماتی مثل مالیات (tax) یا ارزشی مانند میلیون (million) به دلیل هم‌نشینی موضوعی همواره به بردار چنین سندی نزدیک می‌شوند؛ حتی اگر مستقیماً در متن آن سند نوشته نشده باشند.
- **جلوگیری از تداخل کلمات نامرتب:** کلماتی مانند “government”، “help” و “money” که شباهت کسینوسی منفی شدیدی با سند دارند، نشان‌دهنده‌ی مفاهیمی هستند که کاملاً متمایز از محور اصلی این سند بوده و مدل به درستی آن‌ها را پس زده است.

سوال ۱۹ – تحلیل و تصویرسازی رفتار دسته‌ها در فضای پنهان با نقشه‌ی حرارتی (Heatmap)

در این گام، هدف این است که مشخص کنیم آیا فضای پنهان استخراج‌شده توسط SVD توانسته است مرزهای معنایی متمایزی بین دسته‌های مختلف خبری (مانند ورزش، سیاست، تجارت و...) ایجاد کند یا خیر. برای این کار، میانگین بردارهای اسناد متعلق به هر دسته محاسبه شده و به عنوان «امضای معنایی» آن دسته روی نقشه‌ی حرارتی ترسیم می‌شود. کد این بخش به صورت زیر است:

```
1 # =====
2 # Q19 - Mean Latent Space Vector per Category (Heatmap)
3 # =====
4 def q19_category_heatmap(df_train, Uk, Sk, fname):
5     doc_vecs = Uk * Sk
6
7     categories = sorted(df_train['Label'].unique())
8
9     mean_vecs = []
10    for c in categories:
11        mask = (df_train['Label'] == c).values
12        mean_vecs.append(doc_vecs[mask].mean(axis=0))
13
14    mean_vecs = np.array(mean_vecs)
15    print(f"Mean latent vectors per category shape: {mean_vecs.shape}")
16
17    fig, ax = plt.subplots(figsize=(14, 5))
18    sns.heatmap(mean_vecs, cmap='coolwarm', center=0,
19                yticklabels=[LABELS_MAP[c] for c in categories],
20                xticklabels=[f'C{i+1}' for i in range(mean_vecs.shape[1])],
21                ax=ax, cbar_kws={'label': 'Mean latent value'})
22
23    ax.set_title('Mean Latent Space Vector per Category', fontsize=14)
24    ax.set_xlabel('Latent Component', fontsize=12)
25
26    plt.tight_layout()
27    plt.savefig(f'media/{fname}.png', dpi=130)
28    plt.close(fig)
29
30    return mean_vecs
```

توضیح عملکرد متدها و منطق ریاضی:

۱. تشکیل ماتریس اسناد در فضای پنهان: ابتدا با ضرب توزیعی سطرهای ماتریس ویژگی‌های اسناد (U_k) در مقادیر منفرد متناظر آن‌ها (Σ_k)، نمایش واقعی اسناد در فضای کاهش‌یافته محاسبه می‌شود:

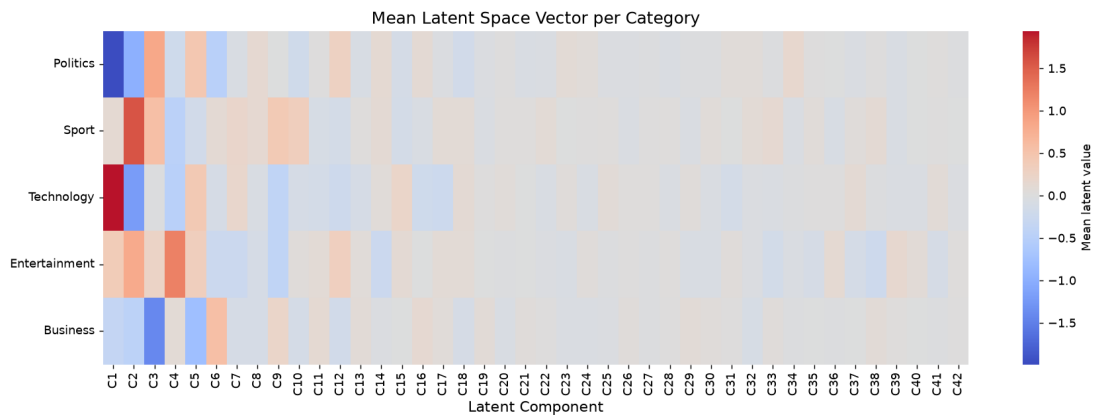
$$D_{\text{latent}} = U_k \cdot \Sigma_k$$

۲. محاسبه‌ی مرکز ثقل دسته‌ها (Centroid Calculation): برای هر دسته خبری c ، یک ماسک منطقی روی داده‌ها اعمال می‌شود تا اسناد متعلق به آن طبقه تفکیک شوند. سپس میانگین هندسی این بردارها روی محور ستون‌ها ($\text{axis}=0$) محاسبه می‌گردد تا یک بردار میانگین متمایز با ابعاد 1×42 به عنوان نماینده‌ی آن دسته حاصل شود:

$$\vec{\mu}_c = \frac{1}{|D_c|} \sum_{d \in D_c} \vec{d}$$

با تجمیع این بردارها برای تمام ۵ دسته، ماتریسی با ابعاد 5×42 تحت عنوان mean_vecs ساخته می‌شود.

۳. رسم نقشه حرارتی با پالت رنگی دوقطبی: نمودار با پالت رنگی coolwarm ترسیم می‌شود که مرکز آن روی عدد 0 تنظیم شده است. مقادیر مثبت (گرم/قرمز) نشان‌دهنده همبستگی مثبت شدید دسته با آن مؤلفه پنهان و مقادیر منفی (سرد/آبی) نشان‌دهنده تضاد یا همبستگی منفی است.



شکل ۱۲: نقشه‌ی حرارتی میانگین بردارهای پنهان به تفکیک ۵ دسته‌ی موضوعی اصلی

تحلیل تجربی نقشه‌ی حرارتی و جدایی‌پذیری دسته‌ها:

پس از اجرای الگوریتم، خروجی نمودار به شکل زیر در پوشه رسانه‌ها ذخیره می‌شود:
با تحلیل دقیق شکل ۱۲، نتایج شگفت‌انگیز زیر از توانایی جداسازی مفاهیم توسط LSI به دست می‌آید:

- **مرزبندی قوی دسته سیاست (Politics) و فناوری (Technology):** مؤلفه اول پنهان (C_1) رفتار کاملاً متضادی را نشان می‌دهد؛ این مؤلفه برای موضوعات سیاسی به شدت منفی (آبی تیره) و برای موضوعات فناوری به شدت مثبت (قرمز تیره) است. این یعنی تنها با بررسی مقدار سند در مولفه‌ی اول (C_1) می‌توان با دقت بالایی اخبار سیاسی را از اخبار فناوری تفکیک کرد.
- **امضای اختصاصی ورزش (Sport):** دسته ورزشی وابستگی شدیدی به مؤلفه دوم (C_2) دارد (رنگ قرمز تیره در سطر دوم). جالب اینجاست که در این مؤلفه، دسته‌های سیاست، فناوری و سرگرمی خنثی یا منفی هستند که نشان می‌دهد C_2 مشخصاً روی مفاهیم ورزشی متمرکز شده است.
- **تمایز حوزه سرگرمی (Entertainment) و تجارت (Business):** موضوعات هنری و سرگرمی بیشترین ثبت مقدار مثبت را روی مؤلفه چهارم (C_4) دارند، در حالی که موضوعات تجاری امضای اختصاصی خود را روی مؤلفه ششم (C_6) با رنگ نارنجی ملایم و همبستگی منفی شدید روی مؤلفه سوم (C_3) نشان می‌دهند.
- **پدیده میرایی مؤلفه‌های فرعی (کاهش واریانس):** اگر به ستون‌های سمت راست نمودار (از C_{20} تا C_{42}) نگاه کنیم، رنگ غالب به سمت خاکستری بی‌اثر (نزدیک به صفر) متمایل می‌شود. این رفتار کاملاً با تئوری SVD همخوانی دارد؛ چرا که مقادیر منفرد به صورت نزولی مرتب شده‌اند و مؤلفه‌های انتهایی واریانس و اطلاعات بسیار کمتری از توزیع داده‌ها را حمل می‌کنند.

سوال ۲۰ – ارزیابی تجربی مدل دسته‌بندی نزدیک‌ترین مرکز ثقل (Nearest Centroid Classifier) در فضای پنهان
در گام پایانی پروژه، یک مدل دسته‌بندی معنایی بر پایه محاسبات هندسی طراحی شده است. این مدل، هر سند جدید از مجموعه‌ی تست (Test Set) را به فضای پنهان منتقل کرده و با سنجش شباهت کسینوسی آن با مرکز ثقل (Centroid) هر یک از دسته‌های پنج‌گانه در داده‌های آموزش، دسته‌ی پیش‌بینی‌شده را تعیین می‌کند. کد این بخش به صورت زیر است:

```
1 # =====
2 # Q20 - Multi-class Classification & Evaluation
3 # =====
4 def cosine_sim_matrix(A, B):
5     A_norm = A / np.linalg.norm(A, axis=1, keepdims=True)
6     B_norm = B / np.linalg.norm(B, axis=1, keepdims=True)
7     return A_norm @ B_norm.T
8
9 def q20_evaluate(df_train, bow_train, df_test, bow_test, scaler, Vtk):
10     X_train_scaled = scaler.transform(bow_train.astype(float))
11     doc_latent_train = X_train_scaled @ Vtk.T
12
13     categories = sorted(df_train['Label'].unique())
14     centroids = np.array([
15         doc_latent_train[(df_train['Label'] == c).values].mean(axis=0)
16         for c in categories
17     ])
```

```

17 ]
18
19 X_test_scaled = scaler.transform(bow_test.astype(float))
20 doc_latent_test = X_test_scaled @ V_k.T
21
22 sims = cosine_sim_matrix(doc_latent_test, centroids)
23 preds = np.array(categories)[np.argmax(sims, axis=1)]
24 true = df_test['Label'].values
25
26 acc_overall = (preds == true).mean()
27 print(f"Overall accuracy: {acc_overall:.4f} (n={len(true)})")
28 for c in categories:
29     mask = true == c
30     if mask.sum() > 0:
31         acc_c = (preds[mask] == true[mask]).mean()
32         print(f" {LABELS_MAP[c]:<15} n={mask.sum():<5} acc={acc_c:.4f}")
33     else:
34         print(f" {LABELS_MAP[c]:<15} n=0 (not present in this test split)")
35 return acc_overall

```

توضیح عملکرد متدها و منطق محاسباتی مدل:

۱. ضرب ماتریسی بهینه‌ی تشابه کسینوسی (cosine_sim_matrix): برای جلوگیری از به‌کارگیری حلقه‌های تکرار کند پایتون، ابتدا سطرهای ماتریس‌های A و B با تقسیم بر نرم اقلیدسی‌شان پیش‌نرمال‌سازی می‌شوند. ضرب ماتریسی حاصل ضرب این دو ماتریس نرمال‌شده ($A_{\text{norm}} @ B_{\text{norm}}.T$)، به طور هم‌زمان تشابه کسینوسی تمام اسناد تست را با تمام مراکز ثقل محاسبه می‌کند.

۲. انتقال داده‌های تست به فضای کاهش‌یافته: با مجهز بودن به ماتریس راست مفرد V_k^T که از داده‌های آموزش استخراج شده است، ابتدا فرکانس اسناد تست با میانگین و انحراف معیار داده‌های آموزش استانداردسازی (scaler.transform) شده و سپس با نگاشت خطی زیر به فضای ۴۲ بعدی منتقل می‌شوند:

$$D_{\text{test_latent}} = X_{\text{test_scaled}} \cdot V_k^T$$

۳. قانون تصمیم‌گیری مرکز ثقل: هر سند تست به دسته‌ای تخصیص می‌یابد که بیشترین شباهت کسینوسی (زاویه نزدیک‌تر به مرکز ثقل در زیرفضای پنهان) را با آن داشته باشد:

$$\hat{y} = \arg \max_{c \in \{1, \dots, 5\}} \text{CosineSimilarity}(\vec{d}_{\text{test}}, \vec{\mu}_c)$$

تحلیل تجربی نتایج دسته‌بندی روی مجموعه داده تست:

پس از اجرای نهایی ارزیابی روی ۲۲۵ سند تست، نتایج زیر حاصل شده است:

```

Overall accuracy: 0.7600 (n=225)
Politics          n=50    acc=0.7200
Sport             n=53    acc=0.9245
Technology        n=38    acc=0.6842
Entertainment     n=36    acc=0.6944
Business          n=48    acc=0.7292

np.float64(0.76)

```

شکل ۱۳: دقت کلی و تفکیک‌شده‌ی مدل دسته‌بندی نزدیک‌ترین مرکز ثقل در فضای پنهان SVD

با تحلیل خروجی ثبت‌شده در شکل ۱۳، کارآمدی هندسی مدل را بررسی می‌کنیم:

- **دقت کلی قابل قبول (Overall Accuracy):** مدل دسته‌بندی بسیار ساده و بدون پارامتر ما موفق شده است به دقت کل ۷۶٪ روی کل داده‌های تست ($n = 225$) دست یابد. این موضوع نشان می‌دهد تقلیل ابعاد به کمک SVD، واریانس‌های مزاحم و نویزهای متنی را حذف کرده و ساختار متمایز هندسی کلمات و اسناد را به خوبی حفظ کرده است.
- **عملکرد درخشان در حوزه ورزش (Sport):** دسته ورزشی با داشتن ۵۳ سند تست به دقت بی‌نظیر ۹۲٪ رسیده است. با تطابق این یافته با نتایج سوال قبل (نقشه حرارتی)، متوجه می‌شویم اسناد ورزشی رفتار به شدت متمرکزی روی مؤلفه دوم (C_2) دارند و مرکز ثقل آن‌ها به شدت در فضای پنهان منزوی و متمایز از سایر دسته‌هاست که طبقه‌بندی آن را تسهیل می‌کند.
- **چالش‌های دسته‌بندی در سایر حوزه‌ها:** کمترین میزان دقت مربوط به دسته‌ی فناوری (Technology) با مقدار ۶۸٪ و سرگرمی (Entertainment) با مقدار ۶۹٪ است. این افت نسبی به علت هم‌پوشانی لغوی برخی مستندات تفریحی و بازی‌های آنلاین با تکنولوژی دیجیتال است که در ابعاد پنهان مرزهای نزدیک‌تری به یکدیگر دارند. با این حال، دستیابی به دقت نزدیک به ۷۰٪ با استفاده از یک مدل پایه مبتنی بر مرکز ثقل، نشان‌دهنده‌ی کارایی بالای ریاضیات جبر خطی در مدیریت معنایی داده‌های متنی است.